Université de Haute-Alsace

Master 1 – UE Réseaux Mobiles – Automne 2026

Cahier des charges

Optimisation des communications V2V par coloration de graphes

Un outil de simulation et d'optimisation d'un réseau véhicule à véhicule, développé en C++.

Groupe	Emir Faycal Benabdallah Moundir Houazar Samy Mohamed Daoudi Salah Eddine Djemaoui
Evaluateur	Laurent Moalic
Formation	Master 1 IM, Université de Haute-Alsace
Document	Version 1.0, à envoyer le 2 octobre 2026
Soutenance	10 décembre 2026

Sommaire

1. Contexte et présentation du projet 4
1.1 Rappel du sujet 4
1.2 Problématique à résoudre 4
1.4 Besoin auquel le projet répond 4
1.5 Public concerné 4
2. Objectifs du projet 4
2.1 Objectif principal 4
2.2 Objectifs secondaires 4
2.3 Résultats attendus 5
2.4 Critères de mesure de la réussite 5
3. Périmètre du projet 5
3.1 Éléments inclus 5
3.2 Éléments exclus 5
3.3 Hypothèses de départ 6
4. Besoins et exigences 6
4.1 Besoins des utilisateurs 6
4.2 Description détaillée des fonctionnalités 6
4.3 Exigences techniques 7
4.4 Exigences de qualité 7
5. Aperçu de l'interface (IHM) 7
6. Livrables attendus 8
7. Méthode de travail 9
7.1 Gestion de projet : méthode agile 9
7.2 Étapes de réalisation 9
7.3 Méthodes de recherche 9
7.4 Outils utilisés 9
7.5 Méthode de collaboration 9
7.6 Organisation des réunions 10
8. Organisation et répartition des tâches 10
9. Planning prévisionnel 11
9.1 Dates clés imposées 11
9.2 Échéancier par sprint 11
10. Ressources et budget 11
11. Contraintes et risques 12
11.1 Contraintes 12

11.2 Risques et solutions envisagées 12
12. Communication et suivi 12
13. Validation et critères de réussite 13
14. Conclusion 13
15. Annexes et références 13
15.1 Bibliographie 13
15.2 Sitographie 13
15.3 Autres annexes 14

1. Contexte et présentation du projet

1.1 Rappel du sujet

Avec l'évolution des voitures connectées, la technologie V2V permettent aux véhicules d'échanger des données directement entre eux, sans intermédiaire. Le problème, c'est que dès que les voitures s'entassent au même endroit, les connexions simultanées saturent le réseau et créent de grosses interférences.

C'est là que notre projet intervient. Le Développent d'une plateforme de simulation capable de modéliser et de représenter ce réseau V2V pour tester différentes stratégies d'allocation des fréquences radio. Concrètement, on va représenter le réseau sous la forme d'un graphe d'interférences dynamique, qui va bouger et se recalculer en temps réel au rythme des déplacements des véhicules.

1.2 Problématique à résoudre

Comment attribuer un canal radio à chaque véhicule afin de limiter les interférences entre véhicules proches, en utilisant le moins de canaux possibles, alors que le réseau est toujours en mouvement et que l'allocation doit être recalculée régulièrement au cours de la simulation ?

Le problème se ramène à une coloration de graphe (problème NP-difficile) : deux véhicules qui sont connectés par une arête doivent recevoir des couleurs différentes. Il faut donc trouver un compromis entre la qualité de la coloration et le temps de calcul.

1.4 Besoin auquel le projet répond

Disposer d'un outil visuel et interactif pour simuler un trafic dense (au moins 2000 véhicules), construire le graphe d'interférences, appliquer plusieurs algorithmes de coloration et comparer leurs performances.

1.5 Public concerné

- Les évaluateurs de l'UE (soutenance).
- Des étudiants ou chercheurs souhaitant expérimenter des stratégies d'allocation de canaux dans un réseau véhiculaire.

2. Objectifs du projet

2.1 Objectif principal

Livrer une application C++ complète qui charge une carte réelle, simule au moins 2000 véhicules, construit le graphe dynamique d'interférences et compare au moins deux algorithmes de coloration.

2.2 Objectifs secondaires

- Affichage fluide (interface réactive) malgré le nombre de véhicules.

- Implémenter trois algorithmes ou plus : glouton, Welsh–Powell, DSATUR, et une recherche locale.
- Étudier la stabilité de l'allocation des canaux au fil du temps (nombre de véhicules qui changent de couleur entre deux recalculs).
- Exporter les résultats (CSV) et produire des courbes de comparaison.

2.3 Résultats attendus

- Un logiciel fonctionnel avec sa documentation (manuel utilisateur/développeur).
- Une étude comparative chiffrée : temps d'exécution, nombre de couleurs, évolution dans le temps, stabilité.

2.4 Critères de mesure de la réussite

Critère	Cible
Nombre de véhicules simulés simultanément	≥ 2000 sans blocage de l'interface
Validité de la coloration	0 conflit : aucune arête entre deux sommets de même couleur
Algorithmes de coloration implémentés et comparables	≥ 2 (objectif : 4)
Fluidité de l'affichage	≥ 25 images/s pour 2000 véhicules (objectif visé, à valider en test)
Temps de construction du graphe + coloration	Nettement inférieur au pas de temps de recalcul (mesuré et documenté)
Contrôle du temps de simulation	Démarrage, pause, accélération, ralentissement, réinitialisation opérationnels

3. Périmètre du projet

3.1 Éléments inclus

- Visualisation interactive d'une zone à partir d'OpenStreetMap (zoom, déplacement, sélection de zone, affichage du réseau routier).
- Construction du graphe routier (intersections, tronçons, sens de circulation, distances).
- Simulation de véhicules (position, vitesse, direction, destination, rayon de transmission).
- Graphe dynamique d'interférences affiché sur la carte.
- Coloration du graphe, comparaison des algorithmes et analyse des performances.

3.2 Éléments exclus

- Simulation physique détaillée de la couche radio (propagation, atténuation, obstacles, normes IEEE 802.11p / C-V2X).

- Simulation microscopique fine du trafic (changements de voie, feux tricolores, priorités) : le déplacement reste simplifié.
- Communication réseau réelle entre machines : tout est simulé en local.

3.3 Hypothèses de départ

– Les données OpenStreetMap d'une zone de taille raisonnable (ville ou quartier) peuvent être exportées puis chargées localement (fichier .osm).
– Tous les véhicules ont la même portée par défaut (paramétrable, éventuellement différente par véhicule).
– Le graphe d'interférences est recalculé à intervalle régulier et non à chaque image affichée.

4. Besoins et exigences

4.1 Besoins des utilisateurs

L'utilisateur doit pouvoir charger une zone, la parcourir, lancer une simulation, régler ses paramètres, observer le graphe et les couleurs sur la carte, puis comparer les algorithmes.

4.2 Description détaillée des fonctionnalités

Les priorités suivent la méthode MoSCoW : M = indispensable, S = souhaitable, C = optionnel.

Étape	Fonctionnalité	Priorité
Étape 1 – Visualisation du territoire	Chargement d'une zone OpenStreetMap (fichier .osm) ou d'un fond de carte Zoom / dézoom (molette), déplacement (clic-glisser) Sélection d'une zone de simulation (rectangle) Affichage du réseau routier	M
Étape 2 – Graphe routier	Nœuds = intersections, arcs = tronçons de route Sens de circulation (routes à sens unique ou double) Distances en mètres calculées sur les coordonnées Filtrage des routes utilisables (exclusion des chemins piétons)	M
Étape 3 – Simulation des véhicules	≥ 2000 véhicules avec position, vitesse, direction, destination, rayon de transmission Déplacement le long des routes vers une destination (plus court chemin), nouvelle destination à l'arrivée Contrôle du temps : démarrer, pause, accélérer, ralentir, réinitialiser Vitesses variables selon le type de route	M (vitesses variables : S)
Étape 4 – Graphe d'interférences	Sommets = véhicules ; arête si distance < portée Recalcul périodique et visualisation des arêtes sur la carte Structure spatiale (grille) pour éviter le calcul de toutes les paires Affichage de statistiques : nombre d'arêtes, degré moyen et maximal	M (grille spatiale : S)

Étape	Fonctionnalité	Priorité
Étape 5 – Coloration et analyse	Algorithmes : glouton, Welsh–Powell, DSATUR, recherche locale Vérification automatique de la validité de la coloration Comparaison : temps d'exécution, nombre de couleurs, stabilité de l'allocation Courbes d'évolution et export CSV des résultats	M (2 algorithmes) ; S (4 algorithmes, courbes) ; C (export)

4.3 Exigences techniques

- Langage : C++17 (ou supérieur), compilation avec CMake, code portable Windows / Linux.
- Interface graphique avec Qt 6 (Qt Widgets, rendu de la carte avec QGraphicsView ou QPainter).
- Lecture des données OSM (XML) avec une bibliothèque légère (Qt XML ou pugixml).
- Simulation et calcul de la coloration séparés de l'affichage, pour garder une interface réactive.

4.4 Exigences de qualité

- Code structuré en modules (carte, graphe routier, simulation, graphe d'interférences, coloration, IHM), commenté et documenté.
- Tests des algorithmes de coloration sur des graphes connus, validité vérifiée automatiquement.
- Gestion du code avec Git ; relecture croisée du code (revue de code) avant chaque fusion.
- Chaque membre comprend et sait expliquer l'intégralité du code intégré au projet, y compris le code produit avec une IA générative.

5. Aperçu de l'interface (IHM)

Esquisse de la fenêtre principale : la carte occupe la zone centrale, avec les véhicules colorés et les arêtes d'interférence ; le panneau de droite regroupe les paramètres et les choix d'affichage ; la barre inférieure contrôle le temps de simulation ; la barre d'état affiche les indicateurs de la coloration en cours.

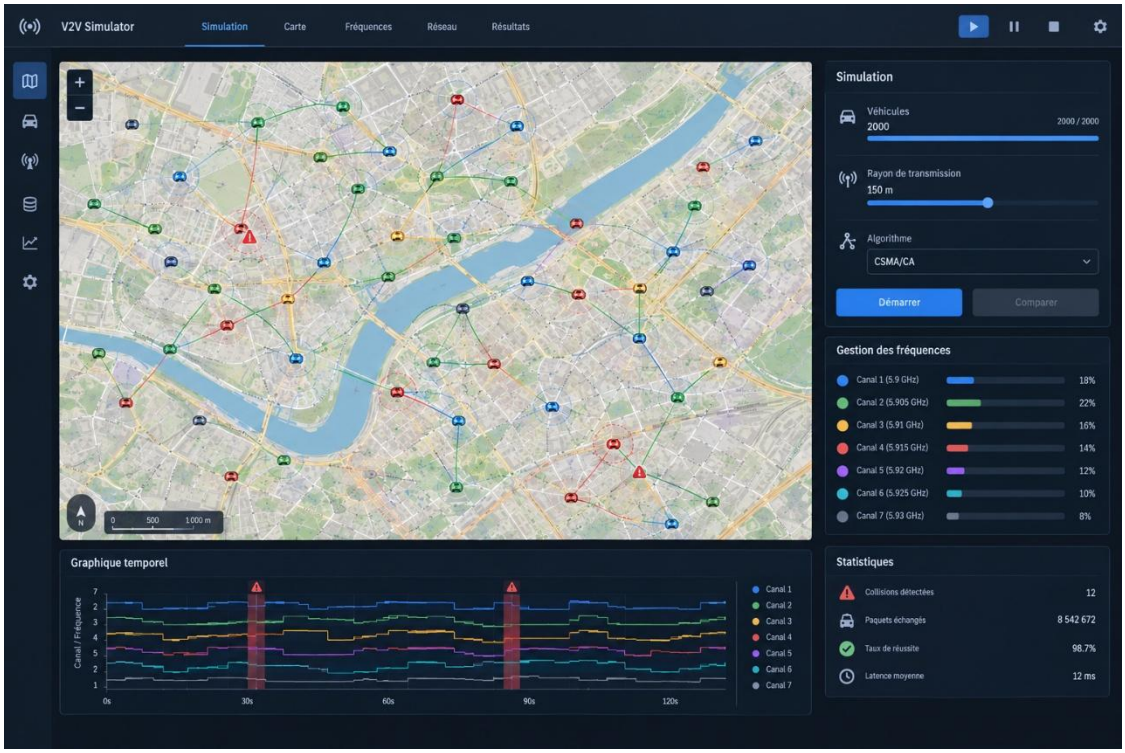


Figure 1 – Maquette de la fenêtre principale

Une seconde fenêtre (ou un onglet) « Comparaison » présentera un tableau et des courbes : nombre de couleurs et temps de calcul par algorithme, au cours du temps.

6. Livrables attendus

Livrable	Contenu	Échéance
Cahier des charges	Le présent document	2 octobre 2026
Slides du point intermédiaire	Démo + quelques slides : avancement, difficultés, planning détaillé de la fin du projet	Semaine du 2 novembre 2026
Sources, données, exécutables	Code C++ complet, jeux de données OSM, exécutable prêt à lancer	Soutenance
Manuel utilisateur / développeur	Section sur l'usage des IA génératives ; architecture du code ; pré-requis ; outils et bibliothèques ; procédure de lancement ; fonctionnalités et utilisation	Soutenance
Package complet	Cahier des charges + slides intermédiaires + sources + manuel	10 décembre 2026
Soutenance orale	Présentation et démonstration du logiciel	10 décembre 2026

Formats : documents remis en PDF (et sources éditables), code accompagné d'un fichier README. Critères d'évaluation : conformité au sujet, qualité du logiciel, pertinence de l'étude comparative, qualité de la soutenance, compréhension du code par chaque membre.

7. Méthode de travail

7.1 Gestion de projet : méthode agile

Nous adoptons une méthode agile de type Scrum allégé : sprints de deux semaines, backlog des fonctionnalités (issues GitHub/GitLab) priorisé selon MoSCoW, et livraison d'une version démontrable à la fin de chaque sprint. Chaque sprint se termine par une courte rétrospective (ce qui a bien marché, ce qu'on améliore). Un tableau Kanban (À faire / En cours / En revue / Terminé) permet de suivre l'avancement au quotidien.

7.2 Étapes de réalisation

Les cinq étapes du sujet sont réalisées dans l'ordre, avec un socle minimal fonctionnel dès le premier sprint (carte + véhicules simples), puis enrichi progressivement. Les algorithmes de coloration peuvent être développés et testés en parallèle sur des graphes aléatoires, sans attendre la simulation.

7.3 Méthodes de recherche

- Étude bibliographique des algorithmes de coloration (glouton, Welsh–Powell, DSATUR, recherche locale).
- Documentation OpenStreetMap (structure des données : nodes, ways, tags « highway », « oneway »).
- Protocole expérimental : plusieurs densités de véhicules, plusieurs portées, plusieurs graines aléatoires ; résultats moyennés.

7.4 Outils utilisés

Domaine	Outil
IDE	Qt Creator et/ou VS Code
Langage et compilation	C++17, CMake, GCC/Clang/MSVC
Bibliothèques	Qt 6 (interface, dessin 2D), pugixml ou Qt XML (lecture OSM), bibliothèque standard C++ (STL)
Données cartographiques	OpenStreetMap (export .osm via le site ou Overpass API)
Versionnement	Git + dépôt distant (GitHub ou GitLab)
Suivi des tâches	Tableau Kanban du dépôt (issues / projects)
Analyse des résultats	Export CSV, tableur ou Python (matplotlib) pour les courbes
Documentation	Word/Markdown pour les documents, commentaires Doxygen pour le code

7.5 Méthode de collaboration

Développement sur des branches Git par fonctionnalité, fusion via demande de fusion (pull request) relue par au moins un autre membre. Convention de nommage et de style de code

définie dès le premier sprint. Le code généré avec une IA est signalé dans le dépôt et relu par l'ensemble du groupe.

7.6 Organisation des réunions

- Une réunion de groupe hebdomadaire (30 à 45 min) : point d'avancement, blocages, répartition des tâches.
- Une séance de planification et une rétrospective par sprint (toutes les deux semaines).

8. Organisation et répartition des tâches

Pour chaque étape, un responsable pilote et répond de l'avancement, et des contributeurs participent. Les noms sont à compléter par le groupe.

Membre	Rôle principal
[Membre 1]	Coordinateur du groupe (planning, relation avec l'encadrant, suivi du dépôt Git)
[Membre 2]	[Ex. : responsable interface et carte]
[Membre 3]	[Ex. : responsable simulation et graphe routier]
[Membre 4]	[Ex. : responsable algorithmes de coloration et analyse]

Étape	Responsable	Contributeurs	Tâches principales
1 – Visualisation	[Membre 2]	[Membre 3]	Lecture des données OSM, affichage, zoom/déplacement, sélection de zone
2 – Graphe routier	[Membre 3]	[Membre 2]	Intersections, tronçons, sens, distances, plus court chemin
3 – Simulation	[Membre 3]	[Membre 1], [Membre 4]	Modèle de véhicule, déplacement, contrôle du temps, optimisation pour 2000 véhicules
4 – Graphe d'interférences	[Membre 1]	[Membre 2], [Membre 4]	Structure de graphe, grille spatiale, affichage des arêtes sur la carte
5 – Coloration et analyse	[Membre 4]	[Membre 1], [Membre 2], [Membre 3]	Algorithmes, tests, comparaisons, courbes, stabilité de l'allocation
Transversal – documentation	[Membre 1]	Tous	Cahier des charges, slides intermédiaires, manuel, package final

Responsabilités individuelles : chaque membre est garant de la qualité et de la documentation de ses modules, et doit pouvoir présenter et expliquer l'ensemble du projet lors de la soutenance.

9. Planning prévisionnel

9.1 Dates clés imposées

Date	Événement
22 septembre 2026	Démarrage du projet
25 septembre 2026	Envoi des groupes et des rôles de chacun
2 octobre 2026	Envoi du cahier des charges
Semaine du 26 octobre	Prise de rendez-vous pour le point intermédiaire
Semaine du 2 novembre 2026	Point intermédiaire (démo + slides)
10 décembre 2026	Soutenance et remise du package complet

9.2 Échéancier par sprint

Période	Sprint / phase	Attendus
22 sept. – 2 oct.	Sprint 0 Cadrage	Groupes et rôles, cahier des charges, dépôt Git, environnement Qt/CMake installé, choix des données OSM
5 – 16 oct.	Sprint 1	Étape 1 terminée (carte, zoom, déplacement, sélection de zone, réseau routier affiché) ; début de l'étape 2
19 – 30 oct.	Sprint 2	Étape 2 terminée (graphe routier) ; étape 3 : véhicules qui roulent, contrôle du temps ; algorithmes de coloration testés sur graphes aléatoires ; préparation du point intermédiaire
2 – 6 nov.	Point intermédiaire	Démo (étapes 1 à 3 au minimum, début de l'étape 4), slides : avancement, difficultés, planning détaillé
9 – 20 nov.	Sprint 3	Étape 3 optimisée (2000 véhicules) ; étape 4 : graphe d'interférences affiché ; étape 5 : glouton + Welsh–Powell + DSATUR intégrés
23 nov. – 4 déc.	Sprint 4	Recherche locale ; comparaison complète, stabilité, courbes, export CSV ; corrections de bugs ; gel des fonctionnalités le 4 décembre
7 – 9 déc.	Finalisation	Manuel utilisateur/développeur, section sur l'usage des IA, package complet, répétition de la soutenance
10 déc.	Soutenance	Présentation, démo, remise du package

10. Ressources et budget

- Matériel : ordinateurs personnels des membres (Windows/Linux), capables de faire tourner 2000 véhicules simulés ; salles machines de l'université en secours.
- Logiciels : Qt 6 (licence open source), Qt Creator / VS Code, CMake, compilateur C++, Git.
- Ressources documentaires : documentation OpenStreetMap et Qt, articles sur la coloration de graphes (voir annexes), supports de cours de l'UE.

- Compétences nécessaires : C++ orienté objet, algorithmique des graphes, programmation d'interface graphique Qt, notions de réseaux mobiles et d'interférences.
- Budget prévisionnel : aucun (outils et données libres et gratuits).

11. Contraintes et risques

11.1 Contraintes

- Temps : environ douze semaines, avec des échéances fixes (2 oct., semaine du 2 nov., 10 déc.) et d'autres cours en parallèle.
- Matérielles : performances variables des machines des membres.
- Budgétaires : aucune ressource payante.

11.2 Risques et solutions envisagées

Risque	Probabilité	Solution de repli
Interface lente avec 2000 véhicules	Moyenne	Séparer simulation et affichage ; dessiner les arêtes seulement si demandé ; limiter le rafraîchissement ; grille spatiale
Données OSM trop volumineuses ou complexes	Moyenne	Zone de taille réduite ; filtrage des routes (« highway ») ; jeu de données pré-découpé fourni avec le projet
Difficulté à afficher un fond de carte (tuiles ou orthophotos)	Moyenne	Afficher uniquement le réseau routier tracé à partir des données OSM
Recolorations qui changent beaucoup les canaux d'une itération à l'autre	Élevée	Conserver les couleurs précédentes comme point de départ (coloration incrémentale) ; recalcul moins fréquent
Retard sur une étape	Moyenne	Priorités MoSCoW : abandon des fonctions « souhaitables » avant les « indispensables » ; entraide entre membres
Indisponibilité d'un membre	Faible	Code relu par au moins deux personnes ; documentation au fil de l'eau
Code généré par IA mal compris	Moyenne	Relecture collective, tests, commentaires ; chaque membre explique ses modules

12. Communication et suivi

- Outils de communication : [Discord / WhatsApp / Teams] pour les échanges courants, e-mail pour l'encadrant.
- Fréquence des réunions : hebdomadaire ; planification et rétrospective toutes les deux semaines.
- Suivi de l'avancement : tableau Kanban, issues, jalons (milestones) alignés sur les sprints.
- Partage des documents : dépôt Git pour le code ; dossier partagé [Drive / OneDrive] pour les documents.

- Gestion des problèmes et retards : tout blocage est signalé au groupe sous 48 h ; en cas de retard important, replanification à la réunion suivante et information de l'encadrant.

13. Validation et critères de réussite

- Personne chargée de valider : Laurent Moalic (jury), lors du point intermédiaire et de la soutenance.
- Critères de conformité : respect des cinq étapes du sujet, de la liste des livrables et du contenu du package final.
- Vérification des objectifs : tests de charge (2000 véhicules), test de validité des colorations, tableau comparatif des algorithmes.
- Corrections éventuelles : intégration des retours du point intermédiaire dans les sprints 3 et 4.
- Conditions de validation finale : application exécutable qui démarre avec la procédure du manuel, démonstration réussie devant l'encadrant, package complet remis le 10 décembre.

14. Conclusion

Ce projet vise à réaliser un simulateur V2V complet en C++, de la carte OpenStreetMap jusqu'à la coloration du graphe d'interférences. Il combine cartographie, simulation, algorithmique des graphes et analyse expérimentale. Grâce à une organisation agile, à des priorités claires et à des solutions de repli identifiées, le groupe s'engage à livrer un logiciel fonctionnel, comparant au moins deux algorithmes de coloration, accompagné d'une étude chiffrée et d'une documentation complète.

15. Annexes et références

15.1 Bibliographie

- D. J. A. Welsh, M. B. Powell, « An upper bound for the chromatic number of a graph and its application to timetabling problems », The Computer Journal, 1967.
- D. Brélaz, « New methods to color the vertices of a graph », Communications of the ACM, 1979 (algorithme DSATUR).
- Supports de cours de l'UE Réseaux Mobiles et sujet du projet – L. Moalic, UHA, automne 2026.

15.2 Sitographie

- OpenStreetMap : https://www.openstreetmap.org (documentation des données : wiki.openstreetmap.org)
- Overpass API : https://overpass-api.de
- Documentation Qt : https://doc.qt.io
- Documentation C++ : https://en.cppreference.com

15.3 Autres annexes

- Usage des IA génératives : une section dédiée sera rédigée dans le manuel final ([à compléter au fil du projet : outils utilisés, parties concernées, vérification]).
- Schémas complémentaires (architecture du code, diagramme de classes) : à ajouter dans le manuel.